UNDERSTANDABLE CONSENSUS

Raft：为了让人看懂而设计的共识算法

Diego Ongaro · John Ousterhout

In Search of an Understandable Consensus Algorithm
Extended Version · Stanford University · 2014-05-20

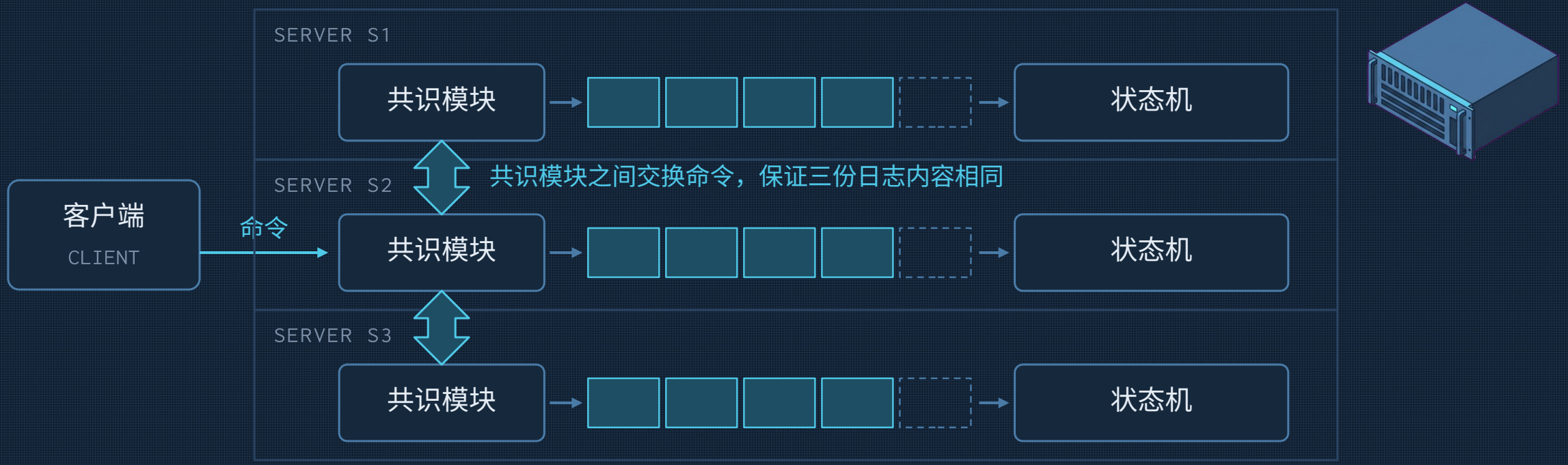
技术深读 · 全文 18 页 · 论文精读



§2 REPLICATED STATE MACHINE

共识算法的工作对象：一份复制日志

每台服务器写同一串命令，状态机按同序执行，集群对外像一台高可靠的状态机。



来源： Ongaro & Ousterhout, In Search of an Understandable Consensus Algorithm (Extended Version), §2 与图 1

§3 WHAT IS WRONG WITH PAXOS

Paxos 的两个缺陷：难懂，且不适合当实现的地基

论文说： Paxos 保证了安全性与活性，也支持成员变更，但它有两个显著缺陷。

FLAW 01

异常难懂

完整解释以晦涩著称，读懂的人极少，
而且只有花很大力气才读得懂。
作者自己读了若干简化解释、又设计出
一套替代协议，前后花了将近一年。

FLAW 02

不是好的实现地基

multi-Paxos 没有公认的算法。
Lamport 主要描述单决议 Paxos，多决议
只给了思路草图，细节大量缺失。
各家实现彼此不同，也与草图不同。

「Paxos 算法的描述与真实世界系统的需要之间存在显著的鸿沟……
最终的系统将建立在一个未经证明的协议之上。」

—— Chubby 实现者，论文 §3 引述

来源：论文 §3。“将近一年”与 NSDI 2012 非正式调查同出该节。

§4 DESIGNING FOR UNDERSTANDABILITY

把“可理解性”当成第一设计目标

面对多种可选方案时，作者用两种通用技术做取舍：谁更容易解释，谁就赢。

TECHNIQUE 01

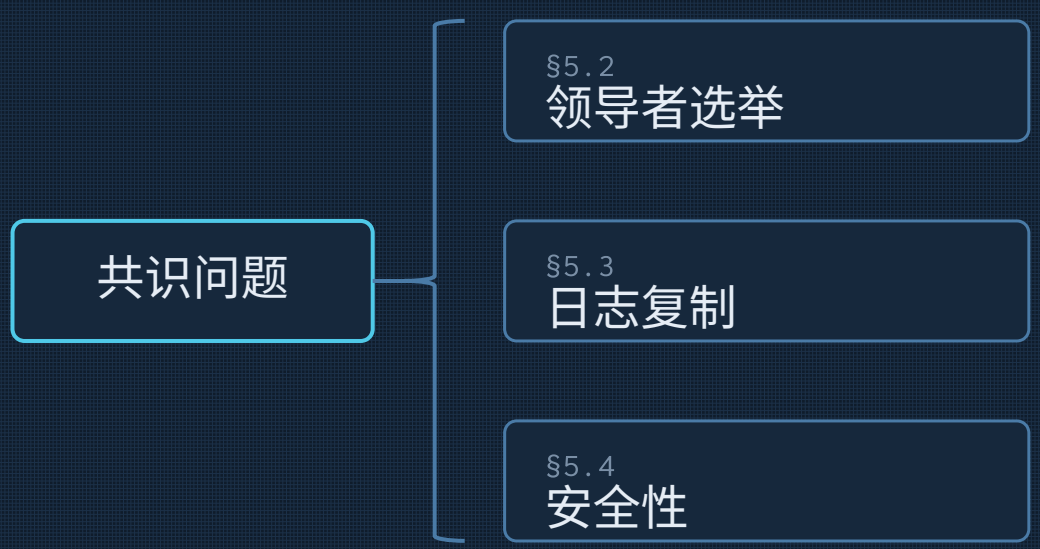
问题分解

把问题切成能独立求解、独立解释、独立理解的几块。

TECHNIQUE 02

缩减状态空间

减少需要考虑的状态数：日志不允许有空洞，不一致的方式也被限制。



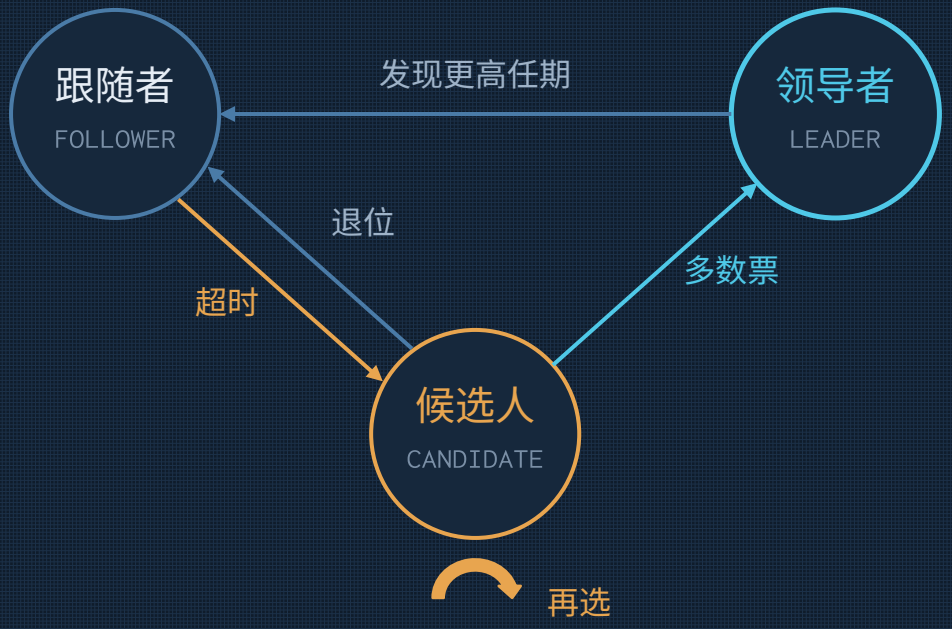
随机化是有意保留的不确定性：“随便选一个，反正都一样”反而缩小了状态空间。

来源：论文 §4（设计取舍与两种技术）与 §5（三个子问题的划分）。

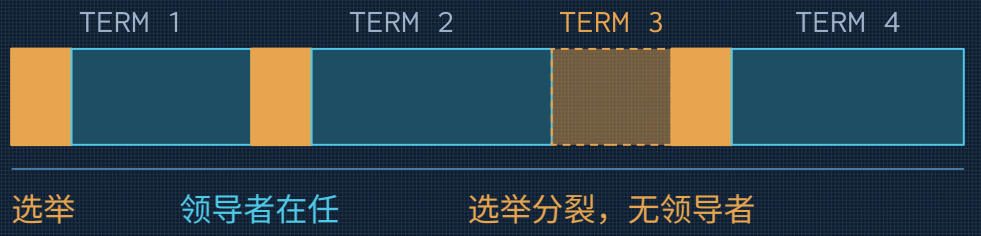
§5.1 STATES AND TERMS

三种服务器状态，一条被切成任期的时间轴

正常运行时恰好一个领导者，其余都是跟随者；任期号则充当整个集群的逻辑时钟。



TERM TIMELINE



任期用连续整数编号，每个任期以一次选举开始；成功则该领导者管到任期末，选举分裂则该任期结束，没有领导者。任期号是逻辑时钟，用来发现过期信息。

来源：论文 §5.1，图 4（服务器状态转换）与图 5（任期）。

§5.2 LEADER ELECTION

选举：心跳、随机超时，和一张先到先得的选票

领导者靠周期性心跳维持权威；心跳一断，跟随者就把自己变成候选人。



1. 跟随者在选举超时内没收到任何通信
2. 自增任期、转为候选人，先投自己一票
3. 并行发出 RequestVote RPC
4. 拿到多数选票 → 成为领导者并发心跳

候选人的三种结局

a 赢得选举：同任期内拿到多数选票

b 别人先确立为领导者，自己退位

c 一段时间无人胜出：票被分裂

每台服务器在给定任期内最多投一票，先到先得。

随机化选举超时

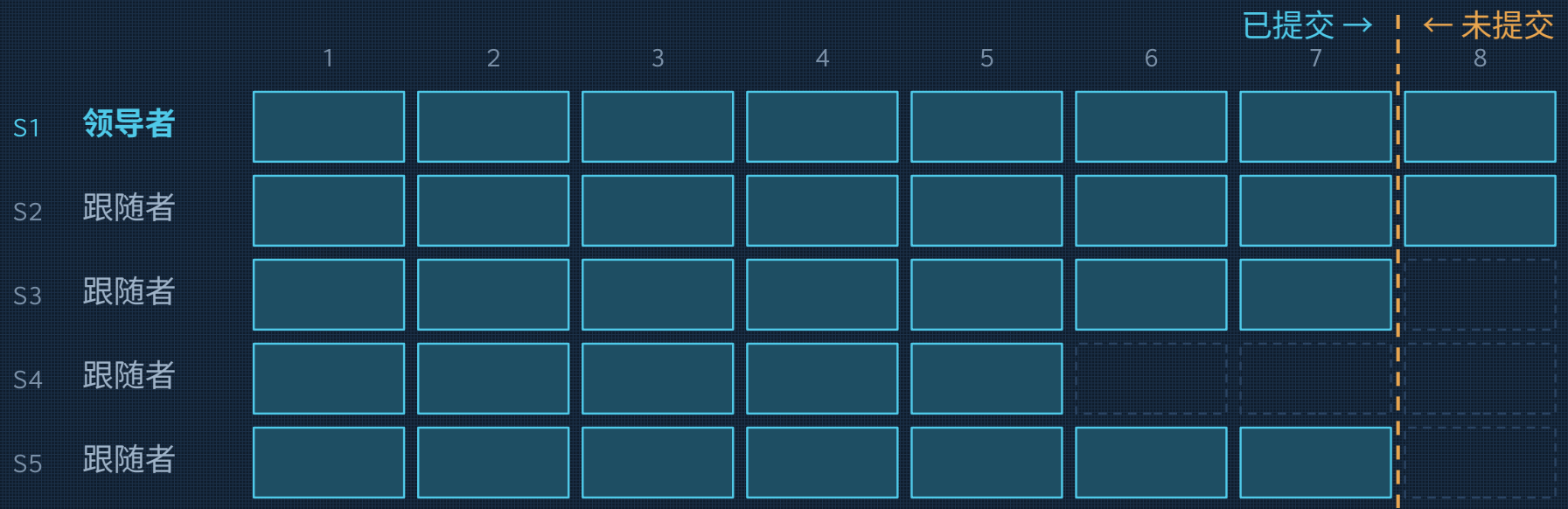
论文举例 150–300ms：多数情况下只有一台服务器先超时，它赢得选举并在别人超时前发出心跳，票分裂因此罕见且能快速化解。

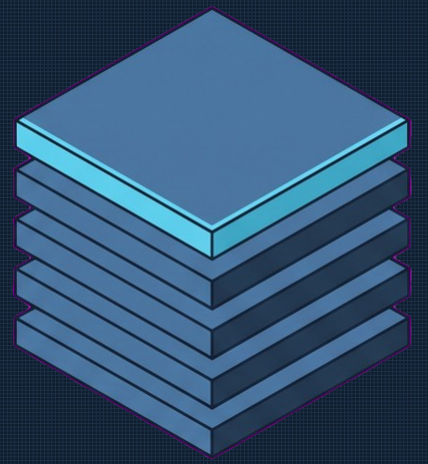
来源：论文 §5.2。心跳即不带日志条目的 AppendEntries RPC。

§5.3 LOG REPLICATION

日志复制：一条命令怎样才算“已提交”

领导者把命令写进日志，并行发出 AppendEntries ，失败就无限重试。





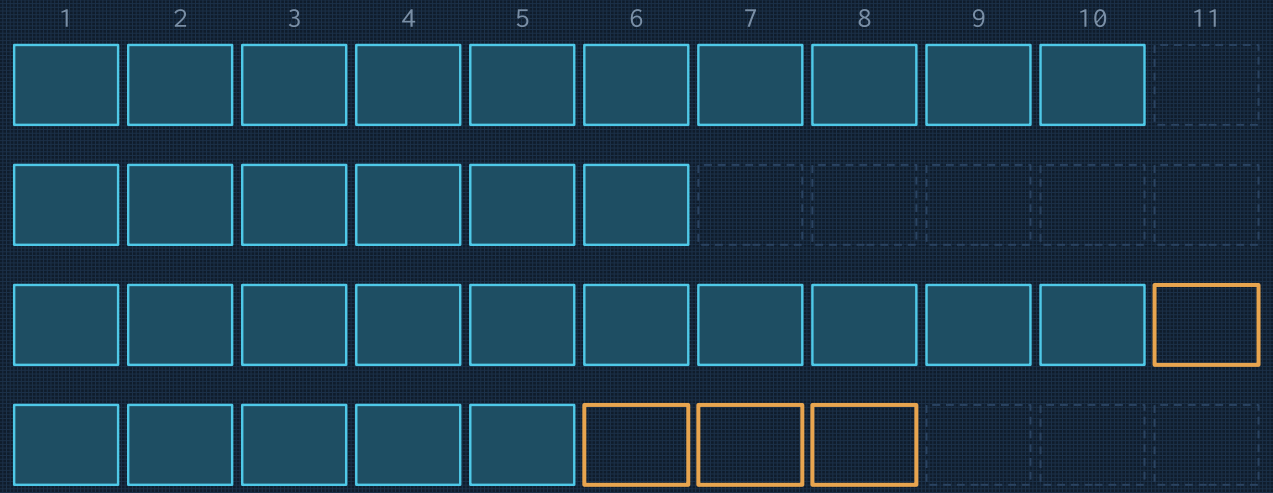
已提交 = 创建该条目的领导者已把它复制到多数服务器上；
这同时提交它之前的所有条目，包括前任领导者创建的那些。

来源：论文 §5.3 。图为按该节提交规则整理的示意（论文图 6 以条目 7 为已提交示例）。

§5.3 LOG MATCHING · FIGURE 7

日志会怎样对不上，以及领导者怎样把它掰回来

日志匹配性质：索引与任期相同的条目，命令相同，且此前的所有条目也完全相同。



nextIndex

领导者上任时，把每个跟随者的 nextIndex 初始化为「自己最后一条日志的索引 + 1」；论文图 7 中这个值是 11 。

- 与领导者匹配
- 冲突（任期不同）
- 缺失

一致性检查失败后，领导者递减 nextIndex 重试；到达匹配点后 AppendEntries 成功，跟随者的冲突条目被删除、领导者的条目被追加。领导者永远不覆盖或删除自己的日志。

> 可选优化：拒绝时带上冲突条目的任期与该任期的首个索引，每个冲突任期只需一次 RPC ；论文认为通常不必要。

来源：论文 §5.3 与图 7 。三行为按图 7 说明整理的示意，论文原图给出 a–f 六种场景。

§5.4 SAFETY

安全性靠两条限制：谁能当选，和什么时候算提交

只有选举和复制还不够：不加限制，新领导者可能覆盖掉已经提交的条目。

§5.4.1 选举限制

候选人必须联系多数服务器才能当选，
因此每个已提交条目至少存在于其中一台上。
RequestVote 带上候选人日志的信息，
投票者若发现自己的日志更新，就拒绝投票。

“更新”的定义

比较两个日志最后条目的索引与任期：任期不同则任期大的更新；任期相同则更长的更新。



§5.4.2 只提交当前任期的条目

领导者只用“计数副本”提交当前任期的条目。
前任任期的条目即使已存到多数，也不算提交（论文图 8）；只有当前任期的条目这样提交后，
此前所有条目才因日志匹配性质被间接提交。

FIGURE 2 · RULES FOR SERVERS / LEADERS

```
If there exists an N such that
N > commitIndex,
a majority of matchIndex[i] >= N,
and log[N].term == currentTerm:
set commitIndex = N
```

来源：论文 §5.4.1、§5.4.2 与图 2（Rules for Servers · Leaders）。

§5 RPC CONTRACT · FIGURE 2

整个基本算法只有两种 RPC

RequestVote 由候选人发起，AppendEntries 由领导者发起；第三种 InstallSnapshot 只用于传快照。

TWO RPCS AT A GLANCE

对照项	RequestVote	AppendEntries
发起者	候选人	领导者
作用	拉票	复制日志 / 心跳
返回	term, voteGranted	term, success
拒绝条件	任期过期或日志不够新	任期过期或 prevLog 不符
论文小节	§5.2 、 §5.4	§5.3 、 §5.2

服务器并行发出 RPC，超时未收到响应就重试；
Raft 的 RPC 是幂等的，重复收到不会造成损害。

APPENDENTRIES · RECEIVER IMPLEMENTATION

```
1. Reply false if term < currentTerm
2. Reply false if log doesn't contain an
entry at prevLogIndex whose term
matches prevLogTerm
3. If an existing entry conflicts with a
new one (same index but different
terms), delete it and all that follow
4. Append any new entries not in the log
5. If leaderCommit > commitIndex, set
commitIndex = min(leaderCommit,
index of last new entry)
```

来源：论文图 2 、 §5.1 、 §5.5 与 §7 。

§6 MEMBERSHIP · §7 LOG COMPACTION

换机器与清日志：两个必须做、又不能出错的运维动作

直接从旧配置切到新配置不安全：切换不是原子的，集群可能分裂成两个各自为政的多数。

§6 联合共识



过渡期用联合共识：日志复制到两个配置的所有服务器；选举与提交都要求旧、新两个配置各自都达到多数。

> 新服务器先作为无投票权成员追赶日志；
> 领导者不在新配置里时，提交 C-new 后退位；
> 被移除的服务器可能捣乱：在最小选举超时内听到过现任领导者，就忽略 RequestVote 。

§7 快照

快照把当前状态写入稳定存储，然后丢弃该点之前的整段日志；每台服务器独立快照，只覆盖已提交条目。
快照另存 last included index / term ，以及截至该索引的最新配置。

领导者对落后太多的跟随者，用 InstallSnapshot RPC 分块发送快照。



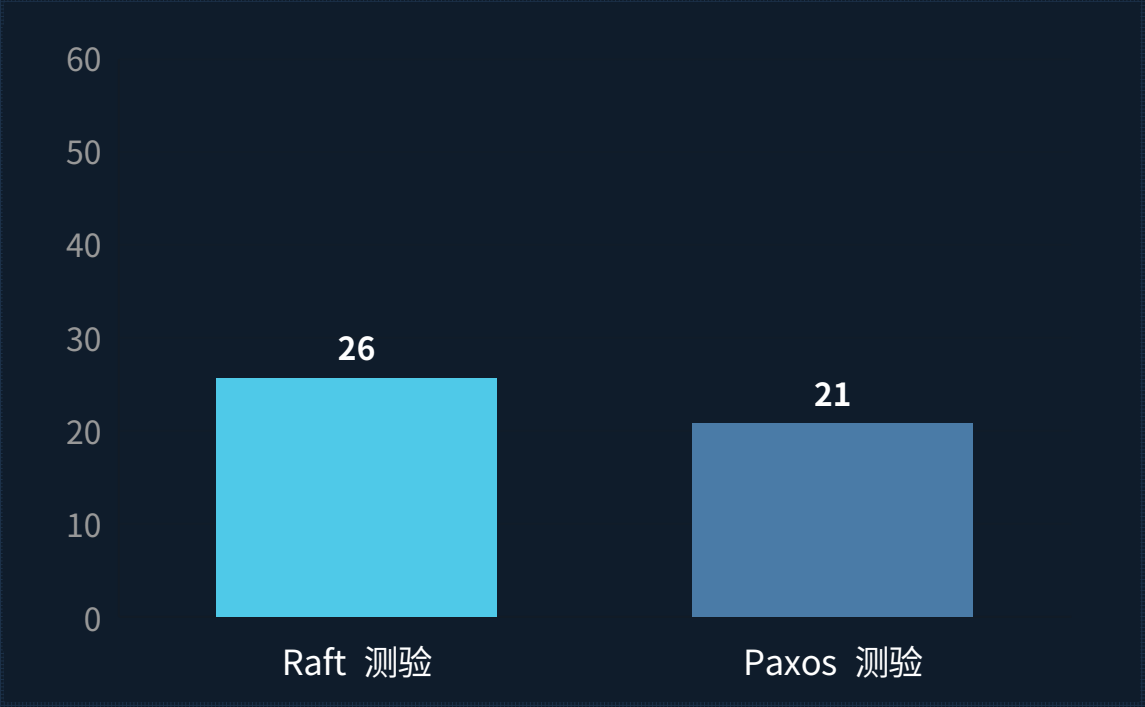
来源：论文 §6（含图 10、图 11）与 §7（含图 12、图 13）。

§9.1 UNDERSTANDABILITY STUDY

把“更好懂”做成了可以测量的东西

43 名学生各看两段视频讲座、各做一次测验；约一半人先学 Paxos，一半人先学 Raft。

MEAN QUIZ SCORE · OUT OF 60



同时学完两种算法后，测验得分更高的一方

33 / 43

平均分（满分 60）

Raft 25.7 · Paxos 20.8

配对 t 检验 · 95% 置信

Raft 的均值至少高 2.5 分

赛后问卷（41 人回答）

33 人认为更易实现，33 人认为更易解释

实验本身偏向 Paxos

15 / 43 有 Paxos 经验；Paxos 视频长 14%

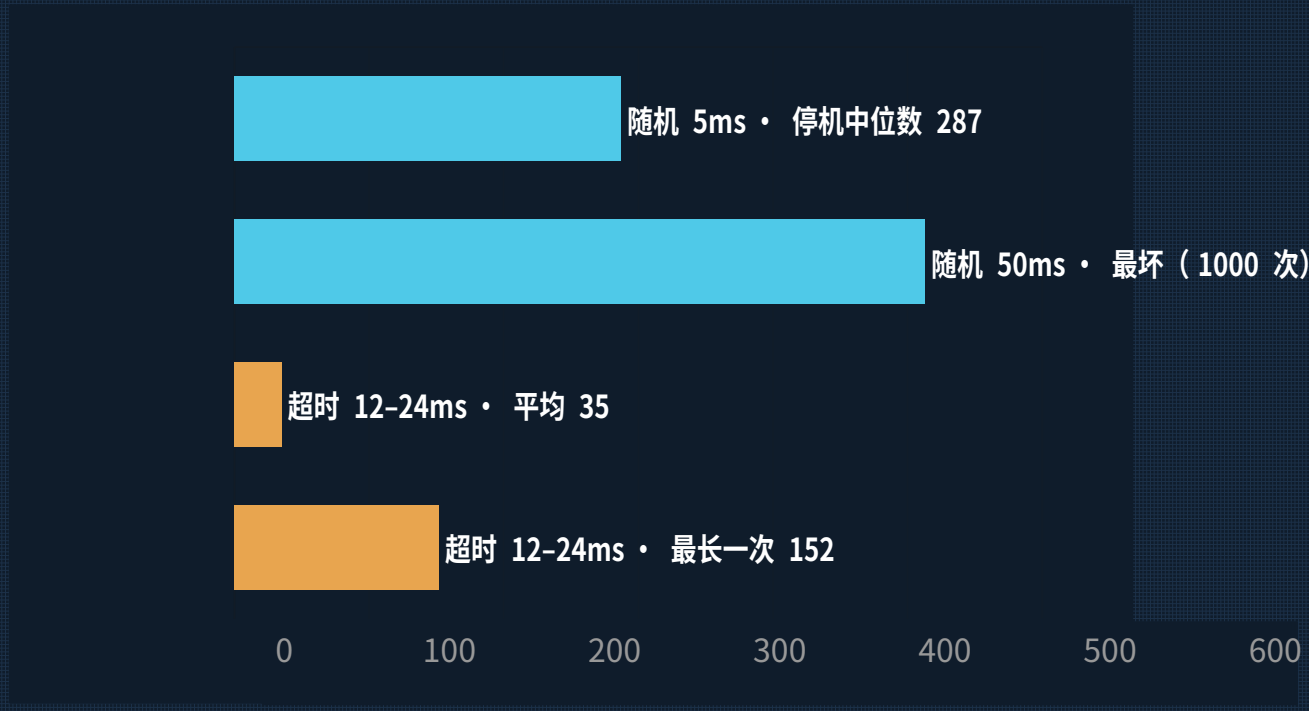
来源：论文 §9.1 正文与表 1；图 14（散点）、图 15（问卷）。数值取自正文，非从图上读取。

§9.3 PERFORMANCE · LEADER ELECTION

一点点随机，就把选举从十秒压到几百毫秒

下面四个数字来自不同的统计口径（中位数 / 最坏 / 平均 / 单次最长），不能互相比较。

TIME WITHOUT LEADER · MILLISECONDS



实验条件

五台服务器集群，广播时间约 15ms；
每条曲线代表 1000 次试验。
领导者在心跳间隔内被均匀随机地崩溃。

随机化的效果

不加随机时选举一直超过 10 秒；
加 5ms 随机，中位停机降到 287ms；
加 50ms 随机，1000 次里的最坏值 513ms。

论文建议

保守取 150–300ms：更低会违反时序要求，带来不必要的领导者更替与可用性下降。

来源：论文 §9.3 正文与图 16（“150–150ms”那条为 100 次试验）。数值取自正文，非从图上读取。

§8 · §9.3 · §10 TRADE-OFFS

Raft 换来的是什么，又付出了什么

论文结论：可理解性与正确性、效率同样重要——开发者不理解算法，就保不住它的性质。

TRADE-OFF 01

强领导者是简化的来源，也是上限

所有客户端请求都经过领导者。 EPaxos 那样的无领导者方案在广域网下延迟更低、负载更均衡，代价是给 Paxos 增加显著的复杂性。

TRADE-OFF 02

安全性不依赖时序，可用性依赖

必须满足 broadcastTime ≪ electionTimeout ≪ MTBF 。领导者崩溃时，系统大约会有一个选举超时那么长的时间不可用。

TRADE-OFF 03

线性一致的只读操作不是免费的

领导者要在任期开始提交一条空的 no-op 条目，并在响应只读请求前与多数交换心跳。改用租约就把安全性押在了时钟偏移上。

TRADE-OFF 04

客户端必须配合去重

命令可能被执行两次：客户端要为每条命令分配唯一序列号，状态机记住每个客户端最近处理过的序列号与响应，重复请求直接返回旧响应。

论文 PDF （扩展版）：
延伸阅读（非论文来源）：官方站与交互式动画演示